

Aterrizaje autónomo de drones en plataformas móviles

Lucio Luque Materazzi, Teo Manuel Kaucher

lluquematerazzi@udesa.edu.ar
tkaucher@udesa.edu.ar

Ingeniería en Inteligencia Artificial
I404 Aprendizaje reforzado

Otoño 2026

Resumen

En este trabajo se desarrolla un entorno de simulación para entrenar un dron cuadricóptero mediante PPO, con el objetivo de aterrizar sobre una plataforma móvil que simula la cinemática de un TurtleBot. Se evalúan distintas configuraciones de recompensa, observación y dificultad progresiva. Los resultados muestran que es posible alcanzar tasas de éxito altas incluso con formulaciones simples, aunque ciertas modificaciones permiten mejorar la estabilidad, la velocidad de aprendizaje o la interpretabilidad del comportamiento.

1. Introducción

Los vehículos aéreos no tripulados (UAVs) cada vez pueden realizar más tareas que antes dependían de humanos, como monitoreo, logística y respuesta ante emergencias. Este crecimiento estuvo acompañado por avances en inteligencia artificial y control autónomo, que permitieron delegar cada vez más decisiones de vuelo al sistema. Sin embargo, dentro de las maniobras que un dron autónomo debe resolver, el aterrizaje sobre una superficie que se mueve es un problema exigente: a diferencia de aterrizar sobre un punto fijo, el agente debe además anticipar y sincronizarse con el desplazamiento del objetivo durante la fase de descenso.

El objetivo de este trabajo es entrenar, mediante aprendizaje por refuerzo (RL), un agente capaz de controlar un dron cuadricóptero para aterrizar de forma autónoma sobre una plataforma en movimiento. Como no existía un entorno de simulación que modelara específicamente este escenario, gran parte del trabajo consistió en desarrollar uno propio, sobre el cual entrenar y evaluar al agente. La meta final de este trabajo es lograr que el dron sea capaz de aterrizar sobre una plataforma cuya cinemática simula la de un robot tipo TurtleBot.

Adicionalmente, uno de los objetivos de este trabajo es sentar las bases para un traspaso de la política entrenada en simulación a un dron físico real (*sim-to-real*), lo cual motivó varias de las decisiones de diseño del entorno de simulación.

A futuro, la intención es extender este trabajo para lograr que el dron sea capaz de aterrizar sobre una plataforma con una dinámica de movimiento más compleja y menos predecible, como una balsa en movimiento sobre agua, la cual debe enfrentar perturbaciones como oleaje, corriente y viento. Si bien el escenario actual no captura estas complejidades, consideramos que es un paso necesario: el entorno, la función de recompensa y las herramientas de entrenamiento desarrolladas en este trabajo son una base sobre la cual extender el problema, permitiendo incorporar de forma incremental la dinámica de la plataforma y las perturbaciones ambientales propias del escenario final, en lugar de enfrentar toda esa complejidad desde cero.

El resto de este informe describe el marco teórico de las técnicas y algoritmos utilizados, el entorno de simulación desarrollado para esta tarea, el proceso iterativo de su implementación, los experimentos abordados y los resultados obtenidos, para finalmente cerrar con las conclusiones del trabajo.

2. Marco teórico

2.1. Curriculum Learning

Curriculum Learning es una estrategia de entrenamiento inspirada en la forma en que los humanos aprenden: presentando las tareas en un orden progresivo de dificultad, en lugar de exponer al agente directamente al problema completo desde el inicio. El concepto fue introducido por Bengio et al. (2009)¹ en el contexto de aprendizaje supervisado, y posteriormente adaptado al aprendizaje por refuerzo.

En RL, el problema de aprender directamente una política sobre la tarea final puede ser ineficiente o complejo cuando la tarea es demasiado exigente desde el principio: el agente rara vez recibe señales de recompensa positivas durante la exploración inicial, lo que ralentiza o impide la convergencia. Curriculum Learning aborda este problema descomponiendo el entrenamiento en una secuencia de tareas $T_1, T_2, \dots, T_n$ de dificultad creciente, de tal manera que la política aprendida en una etapa T_i sirva como punto de partida para la etapa siguiente T_{i+1} .

La dificultad progresiva puede definirse variando distintos aspectos del entorno, como: la posición inicial del agente respecto al objetivo, la presencia de perturbaciones externas (por ejemplo, viento o ruido en los sensores), el movimiento de la plataforma objetivo, etc. Este enfoque permite que el agente adquiera primero comportamientos básicos de control y estabilización, para que después se enfrente a condiciones exigentes, reduciendo así el tiempo total de entrenamiento y mejorando la estabilidad del aprendizaje respecto a entrenar directamente sobre la tarea completa desde el inicio.

En este trabajo, el curriculum se implementa mediante tres ambientes de dificultad creciente: plataforma estática, plataforma con movimiento lineal y plataforma tipo TurtleBot. Estos ambientes se explicarán con mayor detalle más adelante.

2.2. Proximal Policy Optimization (PPO)

PPO es un algoritmo de aprendizaje por refuerzo basado en *policy gradient*, perteneciente a la familia *actor-critic*. Fue propuesto por Schulman et al. (2017)² como una alternativa más simple y estable a TRPO (*Trust Region Policy Optimization*), manteniendo el beneficio de restringir los cambios de la política en cada actualización para evitar pasos demasiado grandes que degraden el desempeño. La idea central de PPO es actualizar la política π_θ maximizando una función objetivo que penaliza aquellos cambios que se alejen demasiado de la política anterior $\pi_{\theta_{old}}$. Esto se logra definiendo:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (1)$$

y la función objetivo:

$$L^{CLIP}(\theta) = \max_{\theta} \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{old}}} \left[\min \left(r_t(\theta) \cdot \hat{A}(s_t, a_t), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}(s_t, a_t) \right) \right] \quad (2)$$

donde $\hat{A}_t$ es el estimador de la función de ventaja y ϵ es el hiperparámetro de clip. Este término permite alcanzar rendimientos similares (o incluso superiores) a TRPO sin necesidad de calcular la matriz de Fisher o la divergencia KL explícita entre las políticas. En nuestra implementación se utilizó $\epsilon = 0.2$, valor estándar recomendado en el trabajo original, que limita a un 20 % la desviación permitida entre la política nueva y la anterior en cada actualización.

PPO combina esta pérdida de política con una pérdida de valor (*critic*) y un término de entropía para fomentar la exploración:

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \quad (3)$$

con la pérdida de la función de valor definida como:

$$L^{VF}(\theta) = \left(V_\phi(s_t) - \hat{R}_t \right)^2 \quad (4)$$

donde $V_\phi(s_t)$ es el valor estimado por el crítico y el retorno real observado $\hat{R}_t$.

Si bien la política π_θ y la función de valor V_ϕ son redes independientes, en la práctica es posible optimizarlas de forma conjunta en un único paso de gradiente, en lugar de realizar dos actualizaciones separadas. Esto se logra combinando ambas pérdidas en una única función objetivo, para que un solo optimizador actualice simultáneamente los parámetros de ambas redes. Esta estrategia simplifica el entrenamiento y reduce el costo computacional, sin alterar el comportamiento teórico del algoritmo.

¹<https://dl.acm.org/doi/10.1145/1553374.1553380>

²<https://arxiv.org/pdf/1707.06347>

3. Entorno de simulación

3.1. Framework utilizado

Para entrenar y evaluar el modelo se usó el entorno `gym-pybullet-drones`³⁴, un simulador basado en PyBullet que provee dinámicas realistas de cuadricópteros y una interfaz compatible con *Gymnasium*.

La elección de este simulador se basó principalmente en el trabajo de Peter et al. (2024)⁵, quienes entrenaron un agente de RL para aterrizaje en plataformas dinámicas utilizando `gym-pybullet-drones`, y lograron transferir exitosamente la política aprendida a un dron real (Crazyflie 2.1). Esto sugiere que el simulador es suficientemente representativo del comportamiento físico real como para permitir el paso sim-to-real, lo cual fue importante para nuestra elección, dado que uno de los objetivos a futuro es evaluar la política entrenada en un dron físico para que estacione sobre un TurtleBot4.

3.2. MovingPlatformLandingAviary

A partir de este simulador se desarrolló un entorno personalizado, `MovingPlatformLandingAviary`, que extiende la clase `VelocityAviary` e incorpora una plataforma de aterrizaje modelada como un cuerpo cinemático que puede desplazarse con velocidad lineal y angular configurables, permitiendo simular tanto plataformas estáticas como en movimiento. En este entorno se utiliza un modelo de dron Crazyflie 2.X.

El entorno permite configurar una gran variedad de parámetros para testear sobre distintos casos (desarrollado en la sección 3.3). Los rangos de velocidad lineal y angular que puede tomar la plataforma en cada episodio $\{(v_{min}, v_{max}), (\omega_{min}, \omega_{max})\}$ se samplean de distribuciones beta $B_v(4,2)$ y $B_w(2,4)$ para generar trayectorias donde el dron no espere a que la plataforma vuelva hacia él. Adicionalmente, se incorporan dos mecanismos independientes para introducir variaciones en el movimiento de la plataforma y acercar su comportamiento al de un TurtleBot4 real: un flag que permite que su velocidad se remuestree dentro del mismo episodio de forma probabilística luego de cierta cantidad de episodios sin cambio, junto con otro flag que agrega ruido a la velocidad en cada paso de simulación. Con respecto al dron, el entorno también permite definir el radio máximo de aparición en el plano (x, y) (r_{xy}) respecto al centro de la plataforma (muestreado de $B_{r_{xy}}(4,2)$ para evitar aterrizajes triviales) y el rango de altura inicial en el eje z (z_{min}, z_{max}) (muestreado con distribución uniforme), ambos seleccionados aleatoriamente al inicio de cada episodio.

Considerando los rangos de las variables mencionadas anteriormente, la duración máxima del episodio en segundos (ep_s) se fija en 20s para permitir la aterrización del dron dentro del desplazamiento esperado de la plataforma, si se mueve a una velocidad aceptable.

Las dimensiones de la plataforma comenzaron formando un plano cuadrado de $30 \times 30 \times 5$ cm para las pruebas iniciales. Luego, teniendo en cuenta el objetivo a futuro del trabajo, se cambió su forma a un cilindro de 25×35 cm. La base del TurtleBot4 sobre la que podría estacionar el dron tiene una forma irregular con un radio máximo de 28cm y mínimo de 25cm, seleccionando el mínimo por seguridad. Además, se fuerza un estacionamiento centrado en la base mediante criterios de estacionamiento y penalizaciones de recompensa (desarrollado más adelante).

En cuanto al vector de observación, inicialmente se intentó incluir únicamente la información necesaria para resolver la tarea: posición relativa del dron respecto a la plataforma (*rel_pos*), velocidad relativa (*rel_vel*), orientación (*roll*, *pitch*, *yaw*), velocidad angular del dron (*drone_ang_vel*) y velocidad absoluta del dron (*drone_vel*), observaciones basadas en las utilizadas por Peter et al. (2024). Durante el desarrollo del trabajo se realizan cambios mínimos para ajustar a términos de recompensa.

El espacio de acciones es continuo y de dimensión 4: las primeras tres componentes definen una dirección de velocidad deseada normalizada ($v_x, v_y, v_z \in [-1, 1]$), mientras que la cuarta componente ($\in [0, 1]$) escala la magnitud de dicha velocidad, ambas heredadas de la interfaz de `VelocityAviary`.

Un aterrizaje se considera exitoso cuando el dron permanece en contacto con la superficie superior de la plataforma, dentro de una tolerancia de posición y velocidad definida, durante un número mínimo de *steps* consecutivos (*stable_counter* ≥ 10). Por otro lado, se considera que colisionó si entra en contacto con algún objeto distinto del tope de la plataforma (costado de plataforma o suelo), o si su ángulo de inclinación supera un umbral crítico (*roll* o *pitch* superan en valor absoluto a 0.8), interpretado como pérdida de control. Como última opción de finalización de episodio, el truncamiento ocurre al exceder ep_s .

³⁴<https://github.com/learnsyslab/gym-pybullet-drones>

⁴<https://arxiv.org/abs/2103.02142>

⁵<https://arxiv.org/pdf/2403.06572>

3.3. Ambientes

Parámetros fijos: en todos los ambientes se utilizó $r_{xy} = 0.8$ y $(z_{min} = 0.5, z_{max} = 1.5)$ para el spawn del dron.

A partir de estos parámetros se definieron tres ambientes de dificultad creciente, utilizados como etapas de *curriculum learning* durante el entrenamiento:

1. **Ambiente estático:** la plataforma permanece fija en su posición inicial durante todo el episodio ($v_{min} = v_{max} = 0, \omega_{min} = \omega_{max} = 0$).
2. **Ambiente lineal:** se selecciona aleatoriamente para la plataforma un ángulo de orientación fijo al inicio del episodio y se desplaza en línea recta a una velocidad lineal constante ($v_{min} = v_{max} \neq 0$), sin rotar ($\omega_{min} = \omega_{max} = 0$). Partiendo de este ambiente, se presentan dos subambientes adicionales: uno en que la velocidad lineal cambia entre episodios ($v_{min} \neq v_{max}$) y otro durante el mismo episodio (`vary_speed=true`).
3. **Ambiente TurtleBot:** la plataforma combina velocidad lineal y angular, emulando la cinemática de un robot TurtleBot, cuyos límites físicos son 0.22 m/s de velocidad lineal y 2.84 rad/s de velocidad angular. La configuración final utilizada para entrenamiento y evaluación, `turtlebot_hard_fixed`, extiende ligeramente estos límites reales ($v \in [0.05, 0.3]$ m/s, $\omega \in [0.0, 3.0]$ rad/s) para exponer al agente a condiciones levemente más exigentes que las del robot físico, y habilita tanto la variación de velocidad durante el episodio como el ruido en cada paso de simulación (`vary_speed=true`, `turt_noise=true`).

4. Desarrollo

4.1. Hiperparámetros generales

En este trabajo se utilizó el algoritmo PPO (explicado en la sección 2.2), implementado por *Stable-Baselines3*⁶, con las siguientes características:

- **Arquitectura de red:** se emplearon dos redes multicapa (*MlpPolicy*) independientes de 2 capas ocultas con 128 neuronas cada una, una para la política (actor) y otra para la función de valor (crítico), sin compartir parámetros entre ambas.
- **Estimación de la ventaja:** se utilizó *Generalized Advantage Estimation* (GAE) con $\gamma = 0.99$ (factor de descuento) y $\lambda = 0.95$ (parámetro de suavizado GAE), que permite balancear sesgo y varianza en la estimación de $\hat{A}_t$.
- **Normalización:** las observaciones y recompensas se normalizaron dinámicamente mediante `VecNormalize`, con recorte de observaciones en ± 10 , lo cual estabiliza el entrenamiento al mantener las entradas de la red en un rango numérico consistente.
- **Hiperparámetros de optimización:** se utilizó una tasa de aprendizaje de 3×10^{-4} , un total de 4096 steps por actualización distribuidos entre los entornos paralelos, tamaño de *batch* de 256, y 5 épocas de optimización por actualización (*n_epochs*). En un principio, `batch_size` era 256 y `n_epochs` era 10, pero se cambiaron sus valores para conseguir un entrenamiento más estable luego de ver explosiones de divergencia KL en algunos entrenamientos.
- **Coficiente de entropía:** se fijó $c_2 = 0.01$ para incentivar la exploración durante el entrenamiento, evitando una convergencia prematura hacia políticas deterministas.

4.2. Pruebas e iteraciones

A lo largo del desarrollo se probaron distintas funciones de recompensa, buscando balancear la velocidad de convergencia del entrenamiento con la calidad y suavidad del comportamiento final del agente.

Primero se definieron las recompensas en los estados terminales. Fueron cambiando en las primeras pruebas, pero se encontró que +25 para aterrizaje, -10 para choque y -2 para truncamiento fueron valores razonables y estables para el entrenamiento. Luego, para reforzar aterrizajes centrados, se incluyó una penalización de $-50 \cdot d_{xy}$ a ese estado terminal.

Se comenzó por una recompensa simple incluyendo distancia a la plataforma y diferencia de velocidad entre la plataforma y el dron. Este enfoque funcionó rápidamente para el ambiente estático, pero tuvo

⁶<https://stable-baselines3.readthedocs.io/en/master/>

problemas con los ambientes lineales de $v_{min} = v_{max} > 0.17$. Este mismo resultado (estático fácil, lineal difícil por sobre una velocidad) se repitió consistentemente en diversas pruebas de recompensa y redujo el progreso del trabajo.

En el siguiente período se revisaron y se cambiaron propiedades y condiciones de la simulación: se cambió la plataforma cuadrada por el cilindro, se arreglaron bugs de distancia del dron calculada hacia el centro de la plataforma en vez de la cara superior, bugs de choque del dron con las caras laterales del cilindro contados como estacionamiento y se cambiaron las condiciones de estacionamiento a valores más estrictos. También se realizaron los cambios en los hiperparámetros mencionados en la sección 4.1. Además, se decidió evitar el curriculum learning y entrenar al dron directamente sobre el ambiente TurtleBot.

Con todos estos cambios y volviendo a la recompensa más simple posible ($-0.1 * d_{total}$) se logró entrenar al dron sobre el ambiente TurtleBot directamente en 4M de pasos. Teniendo una base estable con la cual comparar (denominada V1), se realizaron diversas pruebas. Se mencionan algunas de las proposiciones que se consideran más relevantes.

Como segundo término, se agregó una penalización sobre la inclinación del dron de la forma

$$r = r - k_{ang} \cdot (roll^2 + pitch^2) \quad (5)$$

para obtener un comportamiento más suave. Se encontró que el aumento del factor k_{ang} puede llegar a reducir el `success_rate` y la velocidad de convergencia sin producir cambios notables de comportamiento.

También se intentó incluir una premiación directa asociada al progreso realizado en cada paso, definido como la reducción de la distancia total respecto al paso anterior:

$$r = r + k_p (d_{total}^{prev} - d_{total}) \quad (6)$$

Esta formulación se basa en el enfoque propuesto por Rodriguez-Ramos et al. (2019)⁷, quienes definen su recompensa como la diferencia entre el valor de una función de shaping en pasos consecutivos ($shaping_t - shaping_{t-1}$), premiando así la reducción de la distancia, velocidad y esfuerzo de control relativos entre el dron y la plataforma en cada paso de tiempo. Se demuestra que este término no sirve como reemplazo de la distancia total.

Adicionalmente, se incorporó una recompensa por el contacto entre el dron y la plataforma denominado k_t , de modo que el estacionamiento no dependiera únicamente de reducir la distancia durante n pasos consecutivos, sino que surgiera como un comportamiento aprendido directamente. Con los pesos adecuados, este término cumplió con la hipótesis planteada.

Luego se probó el enfoque conocido como *velocity tracking*, aplicado en problemas de movimiento de robots cuadrúpedos o autos autónomos, que busca definir y seguir un vector de velocidad ideal en cada paso, en vez de una búsqueda indirecta del vector óptimo que minimiza los distintos términos de reward al mismo tiempo. Para los estudios, se decide partir del vector $pos_{plataforma} - pos_{dron}$, ya que la dirección apunta desde el dron hacia la plataforma y se puede escalar su magnitud para definir la velocidad objetivo. El agente resultante de esta política pura lograba seguir la plataforma desde cerca sin estacionar, por lo que se volvió a agregar el término de distancia absoluta.

Para asegurar que el dron tenga la información suficiente como para seguir al vector de velocidad ideal, se lo incluyó en el vector de observación del agente. Además, se cambió la inclusión de roll, pitch y yaw en la observación por los valores en cuaterniones para evitar errores de discontinuidad.

En los resultados se muestran los mejores agentes a lo largo de todas las pruebas.

⁷<https://doi.org/10.1007/s10846-018-0891-8>

5. Resultados

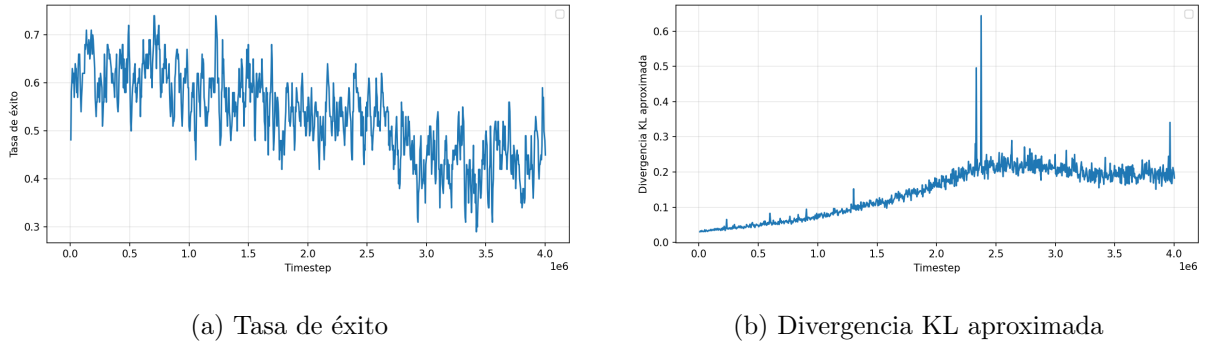


Figura 1: Evolución de la tasa de éxito y de la divergencia KL aproximada durante una configuración inicial de entrenamiento.

La figura 1 muestra un caso de entrenamiento inestable. La tasa de éxito alcanza picos con valores alrededor de 0.7, pero tiende a bajar. En paralelo, la divergencia KL aproximada aumenta de manera progresiva llegando a 0.2 y presenta un pico altamente pronunciado alrededor de los 2.4 millones de timesteps.

Este comportamiento indica que las actualizaciones de la política estaban siendo demasiado agresivas. Una alta divergencia KL permite ver que la política nueva se estaba alejando demasiado de la anterior en cada paso de actualización, yendo en contra del requerimiento principal para que PPO funcione correctamente y generando una pérdida de rendimiento.

A partir de esta observación se modificaron los hiperparámetros de optimización, aumentando el tamaño de bache y reduciendo la cantidad de épocas de entrenamiento por actualización. El objetivo fue realizar pasos de optimización más pequeños y menos ruidosos, preservando mejor la política aprendida.

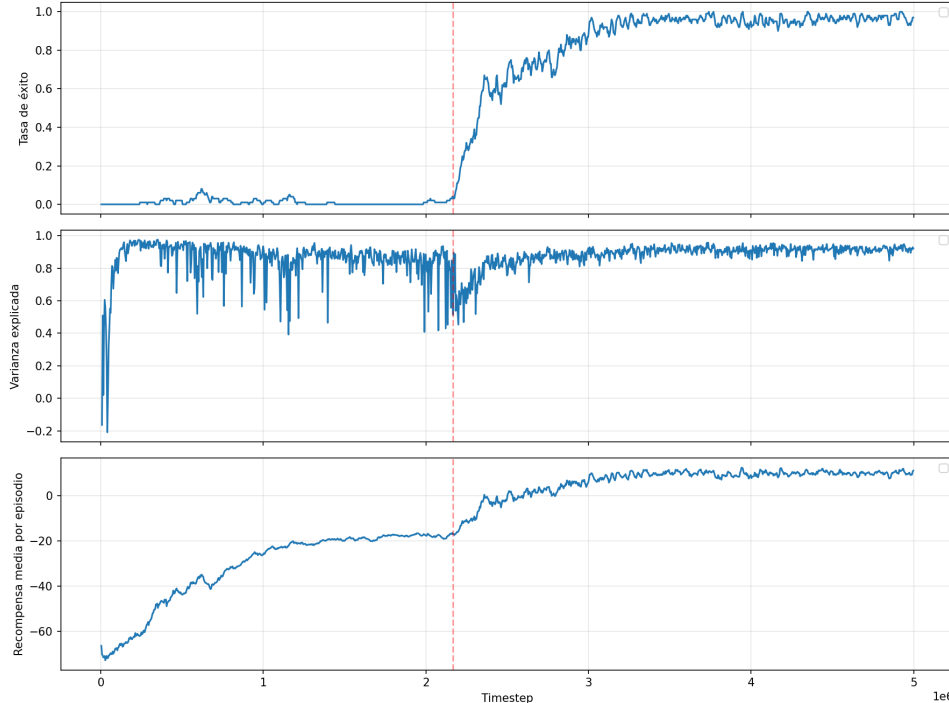


Figura 2: Dinámica de aprendizaje de la versión base V1: tasa de éxito, varianza explicada y recompensa media por episodio.

Habiendo arreglado los hiperparámetros de entrenamiento y algunos errores del entorno simulado, la figura 2 expone distintos gráficos para el agente V1, el primer agente exitoso.

Durante la primera parte del entrenamiento, la tasa de éxito se mantiene cercana a cero, aunque la recompensa media por episodio mejora gradualmente. Esto indica que el dron estaba aprendiendo

comportamientos útiles como acercarse a la plataforma, pero todavía no había descubierto una estrategia completa de aterrizaje.

Luego, alrededor de los 2.1 millones de timesteps, se observa un salto abrupto en la tasa de éxito. Este tipo de cambio es esperable en tareas donde la condición de éxito depende de una secuencia precisa de acciones: acercarse, descender y aterrizar para mantenerse estable durante varios pasos consecutivos. Hasta que el agente no descubre esa secuencia, las mejoras pueden ser lentas o poco visibles en la métrica de éxito. Una vez que la descubre, aprende a repetirla y estacionar de manera más consistente.

La varianza explicada empeora justo en la zona donde la tasa de éxito empieza a crecer. Esto ocurre porque en el momento que el agente pasa de no aterrizar a aterrizar con frecuencia, el crítico debe reajustar sus estimaciones de valor a casos nuevos con recompensas terminales mucho mayores. Después de esa transición, la varianza explicada se recupera y la recompensa media aumenta, mostrando un entrenamiento estable.

Sobre la varianza explicada también se considera relevante remarcar que nunca alcanza un valor de 1 exacto, quedándose sobre 0.95. Esto ocurre porque, debido al comportamiento aleatorio de la plataforma, es muy difícil para el crítico predecir el valor correcto de cada estado.

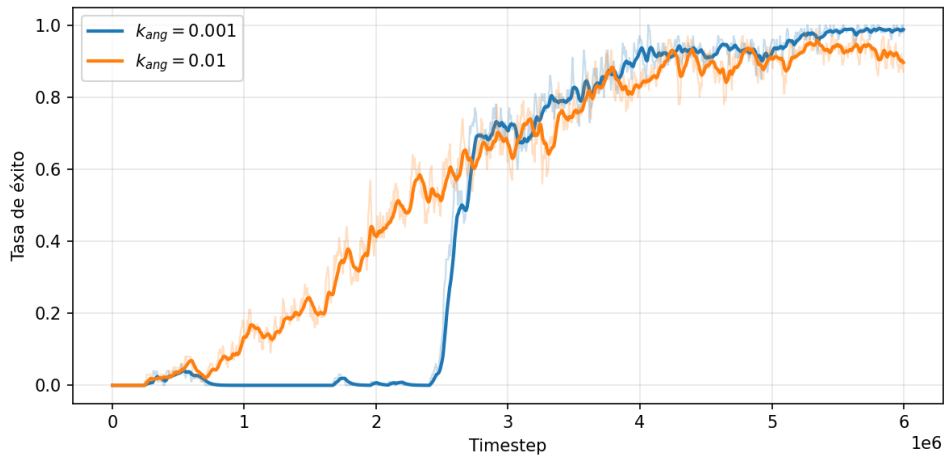


Figura 3: Efecto de la penalización angular k_{ang} sobre la tasa de éxito.

La figura 3 compara dos valores del coeficiente k_{ang} , utilizado para penalizar la inclinación del dron mediante los ángulos de *roll* y *pitch*. El experimento muestra que un valor más alto de penalización angular permite que el agente empiece a estacionar antes: la curva con $k_{ang} = 0.01$ comienza a crecer de manera más temprana y alcanza tasas de éxito intermedias antes que la versión con menor penalización. Se infiere que este comportamiento se debe a las condiciones de estacionamiento: el dron debe tener un ángulo de rotación menor a 0.1 para aumentar el `stable_counter`, un comportamiento causado con mayor frecuencia al aumentar k_{ang} .

Sin embargo, esta ventaja inicial no se mantiene en el resultado final. La versión con $k_{ang} = 0.001$ tarda más en estacionar, pero una vez que lo hace, su tasa de éxito crece de forma abrupta y termina superando a la versión más restrictiva. Una interpretación posible es que una penalización angular fuerte facilita cumplir desde temprano la condición de baja inclinación, pero también limita la capacidad del dron para realizar maniobras correctivas durante la aproximación a una plataforma móvil. En cambio, una política con menor penalización tiene más libertad para inclinarse, acelerar y corregir su trayectoria y tarda más en estabilizarse, pero finalmente alcanza una política más efectiva.

Para evaluar la ventaja conseguida por el término de inclinación, se realiza el siguiente gráfico:

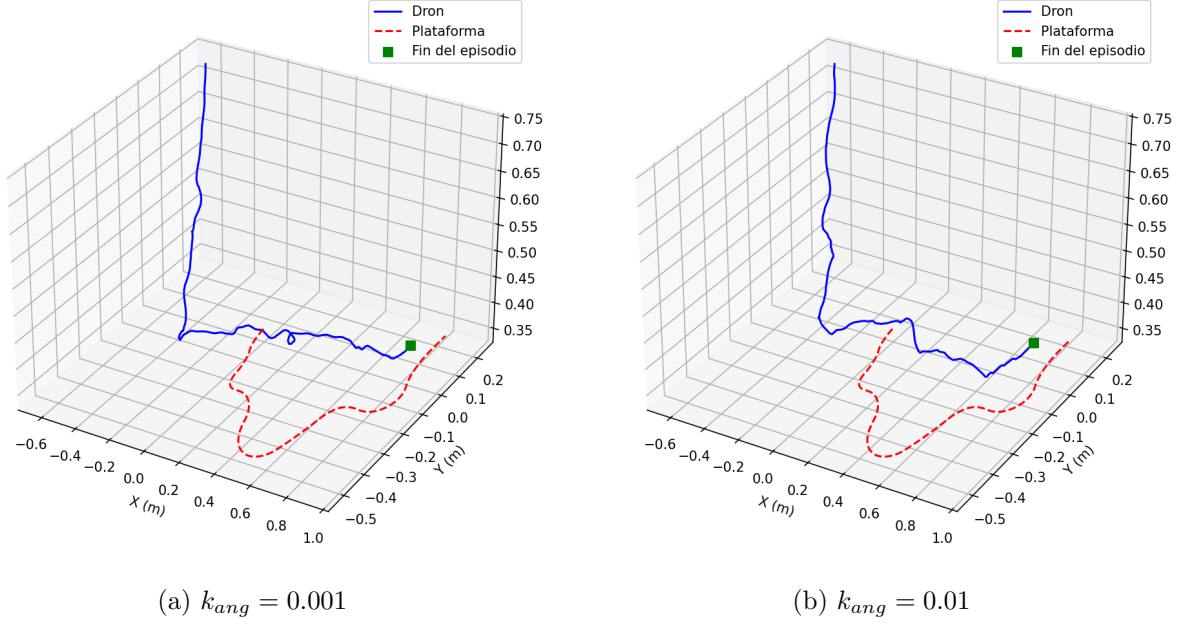


Figura 4: Efecto de la penalización angular k_{ang} sobre la trayectoria.

La figura 4 muestra la trayectoria de los dos agentes para el mismo episodio. Ambos lograron estacionar, a pesar de que el punto verde no se encuentre sobre la línea roja, calculada sobre el centro de la plataforma.

Se puede observar que al aumentar k_{ang} la trayectoria se vuelve un poco más suave, con menos movimientos bruscos. Se considera que el cambio en el comportamiento es demasiado leve como para justificar la pérdida de rendimiento expuesta en la figura 3. Por lo tanto, se continúan los estudios con $k_{ang} = 0.01$.

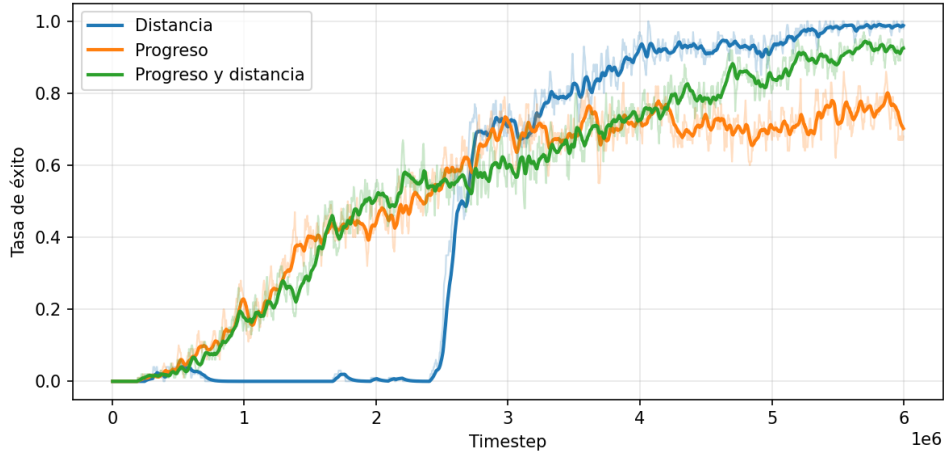


Figura 5: Comparación entre tres formulaciones de recompensa: distancia absoluta a la plataforma, progreso instantáneo y combinación de progreso con distancia.

Se observa en la figura 5 que las versiones que incluyen progreso empiezan a mejorar antes, ya que el agente recibe recompensa por acercamientos parciales a la plataforma. Sin embargo, el progreso por sí solo se estanca en una tasa de éxito menor y presenta mayor oscilación. Esto sugiere que premiar únicamente la mejora local no garantiza que el agente complete la maniobra de aterrizaje: puede acercarse o corregir su trayectoria sin necesariamente descender y estabilizarse sobre la plataforma.

La combinación de progreso y distancia mejora el desempeño respecto del progreso puro, pero tampoco supera a la formulación basada solo en distancia. Aunque esta última tarda más en aprender, una vez que el agente descubre una estrategia efectiva de aterrizaje, la tasa de éxito crece de forma abrupta y alcanza el mejor resultado final. Por este motivo, se decidió conservar la distancia absoluta como término principal de la recompensa.

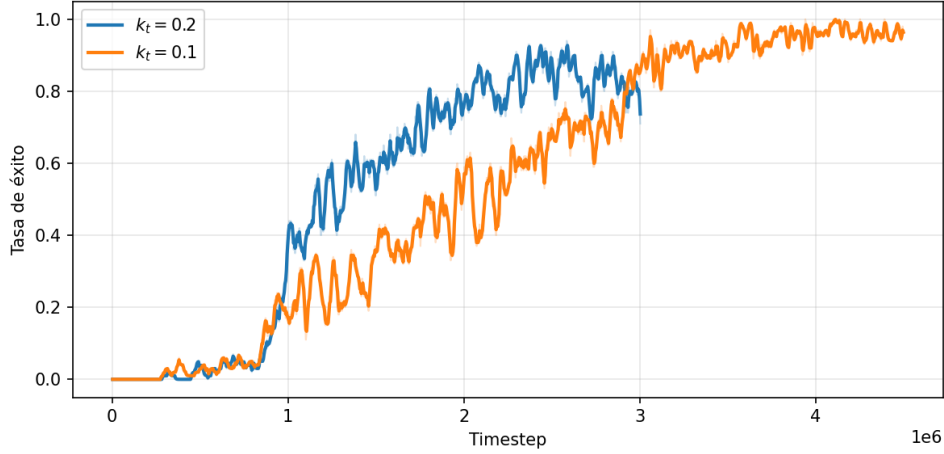


Figura 6: Efecto del peso k_t asociado al contacto con la plataforma.

La figura 6 compara dos valores del término k_t , incorporado para premiar el contacto entre el dron y la plataforma. La motivación de este término fue hacer que el aterrizaje no dependiera únicamente de reducir la distancia, sino que el agente aprendiera explícitamente que tocar la superficie superior y permanecer estable sobre ella era parte central de la tarea.

El valor $k_t = 0.2$ produce una mejora inicial más rápida: la tasa de éxito crece antes y alcanza valores altos en menos timesteps. Sin embargo, la curva presenta mayor variabilidad y no logra consolidar un rendimiento final tan alto como la versión con $k_t = 0.1$. Esto sugiere que premiar demasiado el contacto puede sesgar la política hacia estrategias prematuras o menos precisas, en las que el agente prioriza tocar la plataforma, alejarse y repetir antes de completar correctamente la maniobra de alineación, descenso y estabilización.

En cambio, con $k_t = 0.1$ el aprendizaje es más lento, pero la política termina alcanzando una tasa de éxito superior y más estable. Esta diferencia puede interpretarse como un mejor balance entre los distintos objetivos de la recompensa: acercarse a la plataforma, mantener una velocidad relativa adecuada, descender, tocar la superficie correcta y estabilizarse.

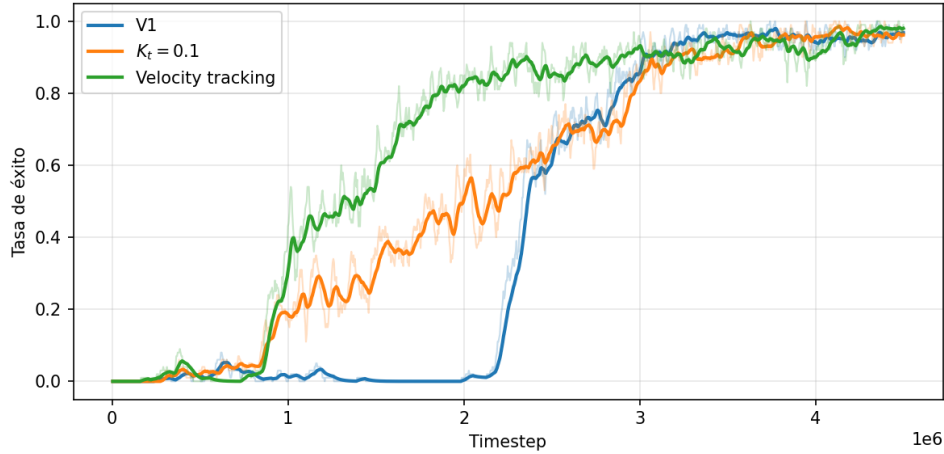


Figura 7: Comparación entre tres configuraciones exitosas: V1, recompensa con contacto $k_t = 0.1$ y *velocity tracking*.

La figura 7 compara tres configuraciones que logran alcanzar una tasa de éxito alta, pero con dinámicas de aprendizaje diferentes. La versión V1 lo logra con la recompensa y observación más mínima posible. Como se mencionó anteriormente, presenta un entrenamiento con resultados tardíos.

La configuración con $k_t = 0.1$ muestra un aprendizaje más progresivo. Al premiar explícitamente el contacto con la plataforma, el agente recibe una señal más directa sobre la acción de aterrizar, lo que evita depender únicamente de la reducción de distancia. Sin embargo, su convergencia es más gradual que la de *velocity tracking*.

La curva de *velocity tracking* es la que mejora más rápido. Esto sugiere que definir una velocidad ideal y exponerla al dron mediante la observación facilita la interpretación del objetivo y orienta al agente

desde etapas tempranas del entrenamiento.

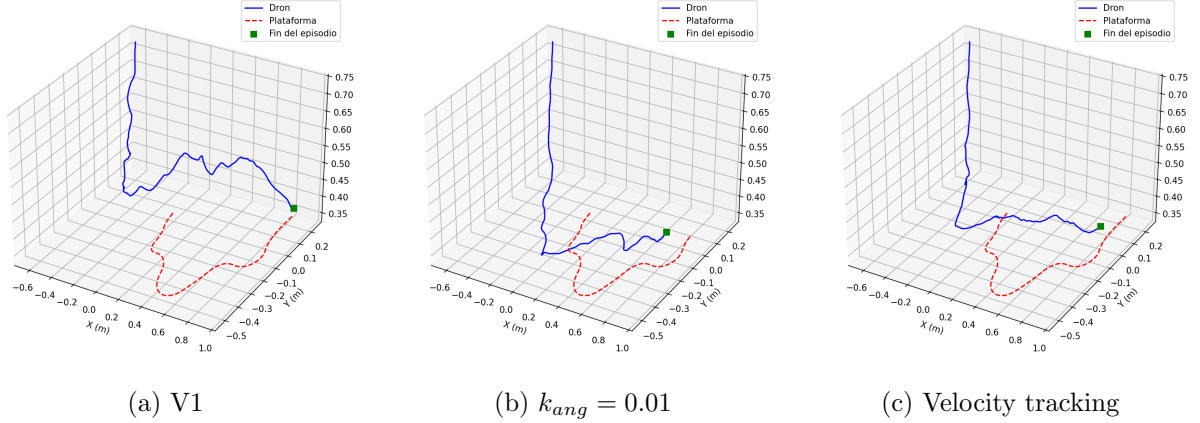


Figura 8: Trayectorias de las tres configuraciones exitosas.

Al analizar la figura 8 se distinguen las diferencias de comportamiento entre los algoritmos: V1 sigue la plataforma de manera ruidosa y poco directa, $k_{ang} = 0.01$ acorta la distancia rápidamente de manera más estable y velocity tracking logra interceptar la plataforma siguiendo el vector ideal.

Considerando ambas figuras, la selección del método final se puede dar por distintos criterios: simplicidad del algoritmo, movimientos suaves y seguros o velocidad e interpretabilidad del entrenamiento.

6. Conclusión

En este trabajo se desarrolló un entorno de simulación para entrenar, mediante aprendizaje por refuerzo, un dron cuadricóptero capaz de aterrizar de forma autónoma sobre una plataforma móvil. A partir de PPO de stable_baselines3 y de un entorno personalizado basado en gym-pybullet-drones, se logró construir un entrenamiento configurable con distintos niveles de dificultad: plataforma estática, plataforma con movimiento lineal y plataforma con cinemática tipo TurtleBot.

Los resultados muestran que es posible alcanzar tasas de éxito altas incluso con una formulación relativamente simple de observaciones y recompensa. La versión base V1 permitió comprobar que una recompensa centrada principalmente en la distancia a la plataforma era suficiente para que el agente descubriera una estrategia efectiva de aterrizaje, aunque con un aprendizaje más tardío y trayectorias menos directas.

Las pruebas con términos adicionales de recompensa permitieron identificar distintos compromisos. La penalización angular favoreció trayectorias algo más suaves y una aproximación más estable, pero no produjo una mejora suficientemente significativa como para justificar siempre la pérdida de rendimiento. Por su parte, el término de contacto con la plataforma ayudó a que el agente aprendiera de manera más directa la acción de aterrizar, aunque valores demasiado altos generaron comportamientos menos precisos y casos de reward hacking.

El enfoque de velocity tracking fue el que mostró una dinámica de aprendizaje más rápida e interpretable, ya que orienta al agente mediante un vector de velocidad ideal hacia la plataforma. Sin embargo, esta mejora se obtiene a costa de agregar mayor información al vector de observación y una gran cantidad de parámetros en la recompensa. En ese sentido, la comparación entre V1, la recompensa con contacto y velocity tracking permite ver que no existe una única configuración óptima: la elección final depende del criterio priorizado, ya sea simplicidad, estabilidad, suavidad de movimiento o velocidad de convergencia.

Si bien el curriculum learning no resultó indispensable para obtener un buen desempeño final, su implementación fue útil durante el desarrollo del entorno. La separación en ambientes de dificultad creciente permitió detectar errores de simulación, ajustar condiciones de éxito, colisión y analizar de forma más controlada cómo respondía el agente ante distintos niveles de complejidad.

Como trabajo futuro, sería importante realizar una búsqueda más sistemática de hiperparámetros y pesos de recompensa, incorporando métodos automáticos de optimización para estudiar con mayor precisión la contribución de cada término. Además, el siguiente paso natural es avanzar hacia la validación sim-to-real, transfiriendo la política entrenada a un dron físico y evaluando su comportamiento para estacionar en un TurtleBot4 real. Finalmente, el entorno podría extenderse hacia el objetivo final: el aterrizaje sobre una balsa móvil, incorporando perturbaciones externas como viento, oleaje, corrientes y movimientos angulares menos predecibles. Por lo tanto, el trabajo realizado constituye una base sólida para seguir investigando maniobras de aterrizaje autónomo en condiciones dinámicas cada vez más realistas.

7. Bibliografía

1. Bengio, Y., Louradour, J., Collobert, R. Weston, J. (2009). Curriculum Learning. Proceedings of the 26th International Conference on Machine Learning (ICML), 41–48. Disponible en: <https://dl.acm.org/doi/10.1145/1553374.1553380>
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. Disponible en: <https://arxiv.org/pdf/1707.06347>
3. Panerati, J., Zheng, H., Zhou, S., Xu, J., Prorok, A. Schoellig, A. P. (2021). gym-pybullet-drones (repositorio de código). Disponible en: <https://github.com/learnsyslab/gym-pybullet-drones>
4. Panerati, J., Zheng, H., Zhou, S., Xu, J., Prorok, A. Schoellig, A. P. (2021). Learning to Fly – a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control. arXiv preprint arXiv:2103.02142. Disponible en: <https://arxiv.org/abs/2103.02142>
5. Peter, R., Ratnabala, L., Aschu, D., Fedoseev, A. Tsetserukou, D. (2024). Lander.AI: Adaptive Landing Behavior Agent for Expertise in 3D Dynamic Platform Landings. arXiv preprint arXiv:2403.06572. Disponible en: <https://arxiv.org/pdf/2403.06572>
6. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research, 22(268), 1–8. Disponible en: <https://stable-baselines3.readthedocs.io/en/master/>
7. Rodriguez-Ramos, A., Sampedro, C., Bavle, H., de la Puente, P. Campoy, P. (2019). A Deep Reinforcement Learning Strategy for UAV Autonomous Landing on a Moving Platform. Journal of Intelligent Robotic Systems, 93(1), 351–366. Disponible en: <https://doi.org/10.1007/s10846-018-0891-8>